

Nakijk 02_01 — V3

NAKIJK V3 · Hoofdstuk 02 · Model, lijst en statuscodes · bij Oefening_02_01.md · derde versie van je code, 9 oktober 2026

Nagekeken: `ProgrammingAdvanced\Oefeningen` → `Controllers\ArtikelController.cs` (gewijzigd op 9 oktober om 23:57). `Models\Artikel.cs`, `Program.cs`, `GlobalUsings.cs` en het projectbestand zijn niet gewijzigd sinds V1.

OPMERKING

Dit is een nakijk van jouw code, geen oplossing. Er staat geen verbeterde code in.

Ik kon je API ook nu niet starten: op de werkruimte bij je laptop staat geen .NET. Ik heb je code gelezen en elk verzoek uit de opgave op papier nagelopen. Wat ASP.NET doet met routes, heb ik nagekeken in de documentatie van Microsoft.

In één oogopslag

Er is veel vooruitgang. De **logica** van punt 4, 5 en 6 doet nu precies wat de opgave vraagt: de 404 voor een onbekende soort werkt, en het nieuwe endpoint voor wat op voorraad is, geeft altijd 200, ook als de lijst leeg is. Je hebt het verschil tussen punt 5 en punt 6 dus goed begrepen.

Er blijft één oorzaak over die bijna alles tegenhoudt: **alle drie je nieuwe routes beginnen met een `/**. Daardoor staan die endpoints niet onder `api/artikel`, maar rechtstreeks aan het begin van het adres. Op het verkeerde adres doet je code dus het juiste. Los je die ene oorzaak op, dan werken punt 4, 5 en 6 in één keer. Dat staat in A1.

Punt	Wat de opgave vraagt	V2	V3
1	het model <code>Artikel</code>	✓ in orde	✓ in orde
2	de controller op <code>api/artikel</code> met minstens zes artikelen	✓ in orde	✓ in orde
3	GET <code>api/artikel</code>	✓ in orde	✓ in orde
4	GET <code>api/artikel/{id}</code>	✗ verkeerd adres	✗ verkeerd adres (A1); de logica is in orde
5	GET <code>api/artikel/soort/{soort}</code>	✗ verkeerd adres, en geen 404	✗ verkeerd adres (A1); de logica is nu in orde
6	GET <code>api/artikel/voorradig</code>	✗ ontbreekt	✗ verkeerd adres (A1); de logica is in orde
7	<code>ArtikelFiche</code> en GET <code>api/artikel/fiche/{id}</code>	✗ ontbreekt	✗ ontbreekt (A4)

Fout uit V2	Status in V3
A1, punt 4: verkeerd adres	nog open, met dezelfde oorzaak als A5
A2, punt 5: onbekende soort geeft geen 404	opgelost
A3, punt 6 ontbreekt	het endpoint is er en de logica klopt; enkel het adres nog (A1)
A4, punt 7 ontbreekt	nog open
A5, punt 5: soortroute	nog open, samengevoegd met A1
B1, returntype van <code>GetArtikelSoort</code>	nog open, en nu ook bij <code>GetVoorraad</code>
C, waarschuwing CS8600 en naamgeving	nog open
nieuw	waarschuwing CS0472 en een overbodige controle in <code>GetVoorraad</code> (C)

Eerst: een misverstand uit V2 rechtzetten

In V2 schreef ik bij de derde regel dat ASP.NET de route van de methode achter die van de controller zet, "met een / ertussen". Dat kon je lezen alsof jij die / moet schrijven. Zo was het niet bedoeld, en dat had ik duidelijker moeten zeggen.

BELANGRIJK

De ``/`` tussen de route van de controller en die van de methode zet ASP.NET er zelf tussen. Jij schrijft er geen. Begint de route van een methode tóch met een /, dan betekent dat iets heel anders: "vergeet de route van de controller, dit is het volledige adres".

Zo stelt ASP.NET een adres samen:

De controller levert	De route van je methode	Wat ASP.NET ervan maakt
<code>api/Artikel</code>	geen route	<code>api/Artikel</code>
<code>api/Artikel</code>	een route die niet met / begint	<code>api/Artikel + / + jouw route</code>
<code>api/Artikel</code>	een route die wel met / begint	enkel jouw route; <code>api/Artikel</code> valt weg

Microsoft zegt het zo: *Route templates applied to an action that begin with / or ~/ don't get combined with route templates applied to the controller.*

A. Wat een verkeerd antwoord geeft

A1. Punt 4, 5 en 6: de drie routes beginnen met /

Waar: `ArtikelController.cs`, de attributen op regel 25, 39 en 63.

Toegepast op jouw code:

Methode	Jouw route	Volledig adres nu	De opgave vraagt
<code>GetArtikelId</code>	<code>"/{id}"</code>	<code>{id}</code> , meteen na de poort, zoals <code>localhost:5145/1</code>	<code>api/artikel/{id}</code>
<code>GetArtikelSoort</code>	<code>"/soort/{soort}"</code>	<code>soort/{soort}</code>	<code>api/artikel/soort/{soort}</code>
<code>GetVoorraad</code>	<code>"/voorradig"</code>	<code>voorradig</code>	<code>api/artikel/voorradig</code>

Wat je API nu antwoordt op de verzoeken uit de opgave:

Verzoek	Je API antwoordt nu	De opgave vraagt
GET <code>api/artikel/1</code>	404 Not Found, zonder bericht	200 OK met artikel 1
GET <code>api/artikel/99</code>	404 Not Found, zonder bericht	404 Not Found met <i>We vonden geen artikel met Id 99.</i>
GET <code>api/artikel/soort/Elektronica</code>	404 Not Found, zonder bericht	200 OK met artikel 1 en 2
GET <code>api/artikel/soort/processor</code>	404 Not Found, zonder bericht	404 Not Found met <i>We vonden geen artikelen van de soort processor.</i>
GET <code>api/artikel/voorradig</code>	404 Not Found, zonder bericht	200 OK met de artikelen met een voorraad groter dan 0

En op de adressen waar je endpoints nu echt staan:

Verzoek	Je API antwoordt nu
GET /1	200 OK met artikel 1
GET /99	404 Not Found met je eigen bericht
GET /soort/processor	404 Not Found met je eigen bericht
GET /voorradig	200 OK met vier artikelen, zonder die met voorraad 0

Die tweede tabel is het bewijs: je code is juist, enkel de adressen niet.

Waarom je op de verzoeken uit de opgave een lege 404 krijgt. Geen enkel endpoint luistert op *api/artikel/...*, behalve *GetArtikelen*. Een verzoek naar *api/artikel/1* bereikt je code dus niet. ASP.NET antwoordt dan zelf, met een lege 404.

Een extra gevolg van `"/{id}"`. Een route die enkel uit `{id}` bestaat en met `/` begint, vangt elk adres dat uit één stuk bestaat, voor je hele API. Tik je in je browser een adres zoals */hallo*, dan komt dat bij *GetArtikelId* terecht. Omdat *hallo* geen getal is, antwoordt ASP.NET dan zelf met **400 Bad Request** en een JSON-object met de fout. Dat doet het omdat je controller *[ApiController]* heeft: die kijkt vóór je methode of de waarden uit het adres in je parameters passen.

Richting.

- Lees de kader hierboven nog eens. Welk teken moet er dan weg uit elk van je drie routes?
- Daarna: wat blijft er over van *api/artikel/{id}* als je het stuk weghaalt dat de controller al levert? Doe hetzelfde voor *soort* en *voorradig*.
- Controleer in Scalar: al je endpoints moeten met *api/Artikel* beginnen.

A4. Punt 7: de artikelfiche ontbreekt nog

Verzoek	Je API antwoordt nu	De opgave vraagt
GET <i>api/artikel/fiche/1</i>	404 Not Found, zonder bericht (er is geen endpoint)	200 OK met de fiche van artikel 1
GET <i>api/artikel/fiche/99</i>	404 Not Found, zonder bericht	404 Not Found met <i>We vonden geen artikel met Id 99</i> .

Ook de klasse *ArtikelFiche* staat nog niet in de map Models.

Richting. Alles uit V2 blijft gelden. Kort samengevat:

- **De klasse.** Een eigen bestand in de map Models, net als *Artikel.cs*. De vier gegevens staan in de tabel van punt 7. *Ja of nee* is in C# een *bool*. *Het artikel zelf* betekent dat dat gegeven van het type *Artikel* is.
- **Het endpoint beantwoordt drie vragen.** Twee ervan heb je nu zelf al opgelost:
 1. Bestaat het artikel? Dat doet *GetArtikelId* al, met hetzelfde bericht.
 2. Hoeveel **andere** artikelen hebben dezelfde soort? Dat lijkt op wat *GetArtikelSoort* doet.
 3. Is er een ander artikel van die soort dat goedkoper is?
- **Twee valkuilen.**
 - Het artikel zelf telt niet mee. Krijg je bij een artikel dat als enige van zijn soort is een 1 in plaats van een 0, dan heb je het artikel zelf meegeteld.
 - "Geen enkel ander artikel is goedkoper" betekent dat twee artikelen met dezelfde laagste prijs allebei ja krijgen.
- `IsVoorradig` volgt dezelfde regel als de voorwaarde die je in *GetVoorraad* schreef.
- **Het returntype** van de methode is een fiche, geen artikel.
- **De route** volgt dezelfde regels als in A1. Zet ze meteen juist.

B. Werkt, maar wijkt af

B1. Twee methodes beloven één artikel, maar geven een lijst

Waar: `ArtikelController.cs`, de kop van `GetArtikelSoort` (regel 40) en van `GetVoorraad` (regel 64).

Wat de client krijgt, klopt: een JSON-lijst. Maar het returntype tussen de punthaken zegt dat er **één** `Artikel` terugkomt.

- **Waarom C# het toelaat.** `Ok(...)` aanvaardt elke waarde, en ASP.NET kijkt niet wat erin zit. Er komt dus geen fout en geen waarschuwing.
- **Waarom het toch telt.** Het returntype is een belofte aan wie je API gebruikt. Scalar leest dat type en toont één artikel als voorbeeld van het antwoord, terwijl je een lijst teruggeeft. Wie je code nakijkt, ziet het meteen.

Richting. Vergelijk met `GetArtikelen` op regel 20: dat geeft ook een lijst terug. Kijk wat daar tussen de punthaken staat. Je nieuwe methode `GetVoorraad` heb je waarschijnlijk vertrokken van `GetArtikelSoort`; daardoor is het daar mee gekopieerd.

C. Opkuis, zonder invloed op het antwoord

C1. In `GetVoorraad` staat een controle die nooit iets doet

Waar: regel 76 tot 79.

Twee dingen tegelijk:

- **De voorwaarde is altijd onwaar.** `Count` is een `int`, en een `int` is nooit `null`. Dat is dezelfde fout die je in V2 bij punt 5 had, en die je daar juist hebt opgelost. De compiler geeft opnieuw de waarschuwing *CS0472: The result of the expression is always 'false' since a value of type 'int' is never equal to 'null' of type 'int'?*.
- **De controle is ook niet nodig.** Kijk wat er binnen de `if` gebeurt, en wat er na de `if` gebeurt: twee keer hetzelfde. Een controle heeft enkel zin als het antwoord daarna verschilt. Bij punt 5 verschilt het (404 of 200), bij punt 6 niet (altijd 200). Daarom heeft punt 5 een controle nodig en punt 6 niet.

Richting. Haal weg wat geen verschil maakt. Je endpoint geeft daarna nog exact hetzelfde antwoord, en de waarschuwing verdwijnt.

C2. Wat er bleef staan uit V2

Waar	Wat	Richting
regel 29	Waarschuwing CS8600: <i>Converting null literal or possible null value to non-nullable type.</i> <code>FirstOrDefault</code> kan <code>null</code> geven, maar het type van je variabele zegt dat dat niet kan.	Kijk hoe de les die variabele schrijft in de <code>ProductController</code> van je theorieproject, of lees in je leerboek, hoofdstuk 17, wat een vraagteken achter een type betekent.
regel 8, 43 en 66	Het veld heet <code>Artikelen</code> , de lijsten in je methodes heten <code>artikelen</code> : enkel één hoofdletter verschil. Op regel 45 en 68 staan ze vlak naast elkaar. Velden beginnen in C# met een kleine letter; de les schrijft <code>laptops</code> .	Geef de lijst in elke methode een naam die zegt wat erin zit.
<code>ProgrammingAdvancedOefeningen.http</code>	Verwijst nog naar <code>/weatherforecast</code> .	Weghalen, of vervangen door de verzoeken uit de opgave.
regel 17-18	Twee lege regels na elkaar.	Eén is genoeg.

Waar je de waarschuwingen ziet. In Visual Studio: *View* → *Error List*, met *Warnings* aangezet, of als een groene golflijn onder de code. Na het opkuisen van C1 en regel 29 hoort je `ArtikelController` geen enkele waarschuwing meer te geven.

D. Wat beter is dan in V2

- **Punt 5 geeft nu een 404 voor een onbekende soort.** Je vergelijkt het aantal met het juiste getal. A2 is opgelost.
- **Punt 6 staat erin, en de logica klopt.** Je filtert op een voorraad groter dan 0, en je geeft de lijst altijd terug met 200, ook als ze leeg is. Dat is precies het verschil met punt 5 dat de opgave wil laten zien.
- **Je hebt de aanpak van punt 5 hergebruikt voor punt 6:** een lege lijst, een lus, een voorwaarde. Dat is een goede gewoonte: een patroon dat werkt, pas je toe waar het past.

E. Zelf nakijken

Doe na het oplossen deze verzoeken, in deze volgorde. Een fout is pas opgelost als het antwoord letterlijk overeenkomt met de opgave: statuscode én bericht.

Verzoek	Wat je moet zien	Hoort bij
Scalar openen	alle endpoints beginnen met <code>api/Artikel</code> ; geen enkel endpoint begint met iets anders	A1
GET <code>api/artikel</code>	200 met je zes artikelen	punt 3
GET <code>api/artikel/1</code>	200 met artikel 1	A1
GET <code>api/artikel/99</code>	404 met <i>We vonden geen artikel met Id 99.</i>	A1
GET <code>api/artikel/soort/Elektronica</code>	200 met artikel 1 en 2	A1
GET <code>api/artikel/soort/processor</code>	404 met <i>We vonden geen artikelen van de soort processor.</i>	A1
GET <code>api/artikel/voorradiig</code>	200, en geen enkel artikel met voorraad 0 in het antwoord	A1
idem, met tijdelijk alle voorraden op 0	200 met een lege lijst, geen 404	punt 6
GET <code>/1</code>	404: het oude adres werkt niet meer	A1
Error List na een build	geen waarschuwing meer in <code>ArtikelController</code>	C1, C2
GET <code>api/artikel/fiche/{id}</code>	de drie controles onderaan punt 7 van de opgave	A4

Verzoek	Wat je moet zien	Hoort bij
GET <code>api/artikel/fiche/99</code>	404 met <i>We vonden geen artikel met Id 99.</i>	A4

Met het nakijksript, als je API in Visual Studio gesloten is:

```
.\Nakijk.ps1 -Oefening 02_01 -Project
"... \ProgrammingAdvancedOefeningen \ProgrammingAdvancedOefeningen"
```

Het script start je API zelf en stuurt de verzoeken uit de opgave. Het verslag komt in de map `Nakijken` naast je project.

Vragen die je je misschien stelt

Mijn endpoints werken als ik ze test. Waarom is het dan fout? Omdat je ze test op het adres waar ze nu staan, zoals `/1`. De opgave, en een examiner, roepen `api/artikel/1` op. Test altijd met het adres letterlijk uit de opgave, of open Scalar en vergelijk het adres.

Waarom staat ``GetArtikelen`` wel goed? Omdat het geen eigen route heeft. Het krijgt dan gewoon het adres van de controller, `api/Artikel`. Dat is de eerste rij in de tabel hierboven.

**Waarom geeft ASP.NET een 400 als ik `/hallo intik`, en geen 404? Zie een extra gevolg in A1. Zodra de routes juist staan, valt dat weg: `/hallo*` past dan bij geen enkel endpoint en geeft een gewone 404.*

Waarom is `Count == null`` een waarschuwing en geen fout? C# laat toe dat je een getal met `null` vergelijkt: het behandelt het getal dan even als een getal dat ook `null` mag zijn (`int?`). Zo'n vergelijking is altijd onwaar, en dat zegt de waarschuwing. Ze breekt je build niet, maar wijst wel op een fout in je denkwerk. Lees waarschuwingen dus altijd.

Moet ``ArtikelFiche`` in een eigen bestand? De opgave zegt *in de map Models*. De gewoonte in C# is één klasse per bestand, met dezelfde naam als de klasse, zoals bij `Artikel.cs`.

Moet ``[HttpGet()]`` met of zonder haakjes? Allebei goed. Met lege haakjes betekent het hetzelfde als zonder: geen eigen route.

Waarom moet de lijst vóór de lus staan? Dat stond in V2 bij A2: een variabele bestaat enkel binnen het blok tussen accolades waarin je ze maakt. Na de lus heb je ze nog nodig, dus maak je ze ervoor. Je hebt dat in `GetVoorraad` opnieuw juist gedaan.

Buiten deze oefening

Niet nagekeken, maar opgevallen: in de `OnderdelenController` heet de routeparameter van het codeendpoint anders dan de parameter van de methode. Dat is hetzelfde mechanisme als fout 1 tot 3 in 01_03: de naam tussen de accolades koppelt alleen aan een parameter met precies dezelfde naam.